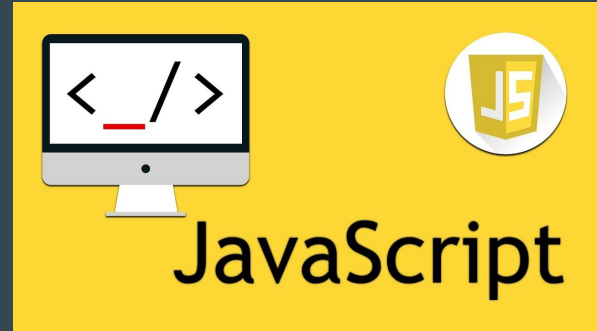


Summarizer



05/04/2023

Ana Jesus



JavaScript

Node

Objects

Data Types

JavaScript

References

Functions Expressions

Hoisting

Primitive Data Types

Arrays

Global Objects

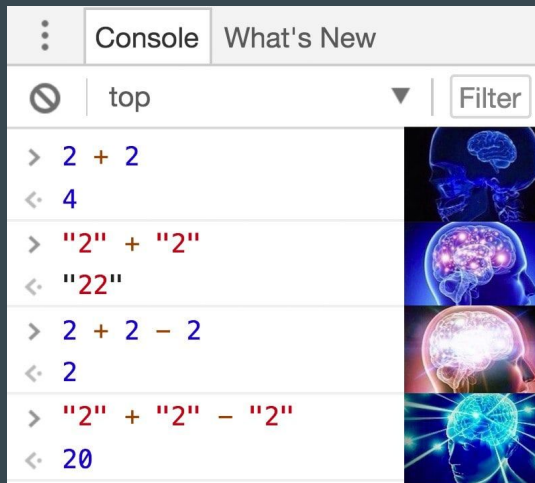
Scope

ECMAScript

JavaScript

Good Stuff

- First-class Functions
- Closures
- Prototypes
- Object literals
- Array literals



Bad Stuff

- Implied globals
- Function scope
- Complex this binding
- Aggressive type coercion
 - Falsy values

Exercise

Green Check Mark all tests

logical operators

- ✓ you should be able to work with logical or
- ✓ you should be able to work with logical and

best practices

- ✓ you should avoid global variables
- ✓ you should use parseInt correctly
- ✓ you should understand strict comparison

flow control

- ✓ you should be able to conditionally branch your code

numbers

- ✓ you should be able to return the decimal representation of a binary string
- ✓ you should be able to add with a precision of 2
- ✓ you should be able to multiply with a precision of 4

arrays

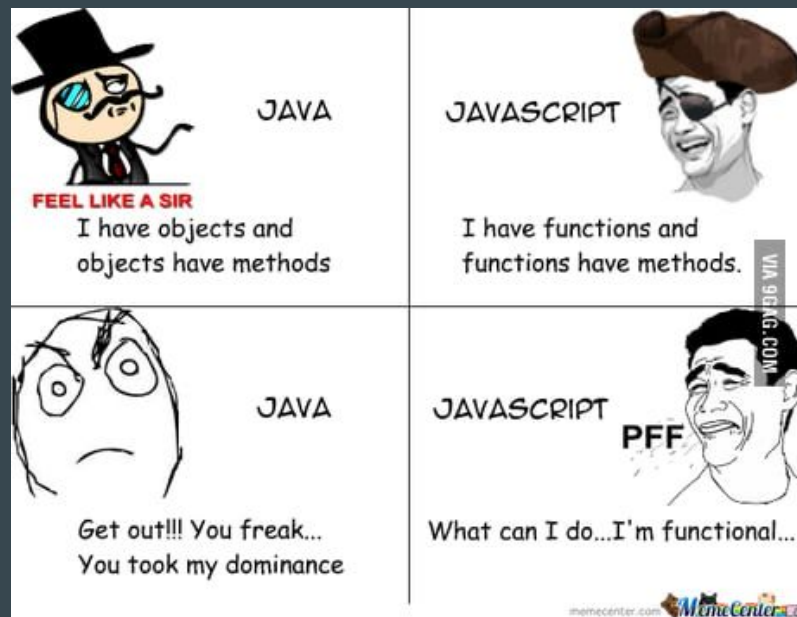
- ✓ you should be able to determine the location of an item in an array
- ✓ you should be able to sum the items of an array
- ✓ you should be able to remove all instances of an item from an array without mutation
- 1) you should be able to remove all instances of an item from an array, returning the original array
- ✓ you should be able to add an item to the end of an array
- ✓ you should be able to remove the last item of an array
- ✓ you should be able to add an item to the beginning of an array
- ✓ you should be able to remove the first item of an array
- ✓ you should be able to join together two arrays
- ✓ you should be able to add an item anywhere in an array
- 2) you should be able to count the occurrences of an item in an array
- 3) you should be able to find duplicates in an array
- 4) you should be able to square each number in an array
- 5) you should be able to find all occurrences of an item in an array

strings

- 6) you should be able to reduce duplicate characters to a desired minimum
- 7) you should be able to wrap lines at a given number of columns, without breaking words
- 8) you should be able to reverse a string
- 9) you should be able to check if string is palindrome

18 passing (49ms)

9 failing



Best Practice

```
2  * Best Practices
3  */
4
5  /**
6   * Obtain user details
7   */
8  //FIXME: there is an issue with this implementation, please fix it
9  exports.getUser = function() {
10     object = {
11         name: 'Pedro',
12         email: 'pedro.antoninho@academiadecodigo.org'
13     };
14
15     return object;
16 };
17
18 /**
19  * Convert String to Number
20  */
21 exports.parseNumber = function(num) {
22     return parseInt(num, 10);
23 };
24
25 /**
26  * Tests for equality
27  */
28 exports.isEqual = function(val1, val2) {
29     return val1 === val2;
30 };
```

Flow Control

```
* write a function that receives a number as its argument;
* if the number is divisible by 3, the function should return 'fizz';
* if the number is divisible by 5, the function should return 'buzz';
* if the number is divisible by 3 and 5, the function should return 'fizzbuzz';
*
* otherwise the function should return the number, or false if no number
* was provided or if the value provided was not a number
*/
exports.fizzBuzz = function(num) {

    if(num % 3 == 0 && num % 5 == 0){
        return 'fizzbuzz';

    }else if (num % 5 == 0){
        return 'buzz';

    }else if (num % 3 == 0){
        return 'fizz';

    }else if(isNaN(num)){
        return false;

    }else{
        return num;
    }

};
```

Numbers

```
* Convert a binary String to a Number
*/
exports.binaryToDecimal = function(str) {
    return parseInt(str, 2);
};

/**
 * Add two Numbers with a precision of 2
 */
exports.add = function(a, b) {
    return Number((a+b).toFixed(2));
};

/**
 * Multiply two Numbers with a precision of 4
 */
exports.multiply = function(a, b) {
    return Number((a*b).toFixed(4));
};
```

Arrays

```
/**
 * Determine the location of an item in the array
 */
exports.indexOf = function(arr, item) {
  return arr.indexOf(item, arr);
};

/**
 * Sum the items of an array
 */
exports.sum = function(arr) {
  var item = (accumulator, curr) => accumulator + curr;
  return arr.reduce(item);
};

/**
 * Remove all instances of an item from an array
 */
exports.remove = function(arr, item) {
  return arr.filter(a => a !== item)
};

/**
 * Remove all instances of an item from an array by mutating original array
 */
exports.removeWithoutCopy = function(arr, item) {
  arr.filter(a => a !== item)
  return arr
};
```



Thank you